

ELF-DOS User's Guide

ELF-DOS is a disk operating system for the CDP1802 microprocessor. It uses the FAT16 file system, the same one used by early versions of MS-DOS, so disks made on ELF-DOS can be read on most other computers, and vice versa.

This guide explains how to use ELF-DOS from day to day: starting the computer, typing commands, working with files and directories, and writing batch files. If you want to know how ELF-DOS works on the inside, or how to write your own programs for it, see the *ELF-DOS Developer's Guide*.

Starting ELF-DOS

When you turn on the computer, ELF-DOS prints a short banner and shows you a prompt, such as:

```
C: />
```

The letter before the colon is the current drive. At start-up ELF-DOS reads the partition table of the drive it booted from and makes each of its partitions available, lettered **C:** through **F:** in order. The text after the colon is the current directory on that drive. If your current directory is more than one level deep from the root, the first part of the path is replaced by **...**; use the **PWD** command to see the full path.

Those four are only a starting point. Six drives can be in use at once, and a drive can carry any letter from **A:** to **Z:** - see "Drives" below.

To switch to a different drive, type its letter followed by a colon, and press Enter:

```
D:
```

The **CD** command changes the current directory. Typing **CD** by itself with just a drive letter (**CD D:**) does not switch you to that drive. It shows or changes D:'s own current directory without leaving the drive you are on. To actually change drives, type the drive letter alone as shown above.

If a file named **autoexec.bat** exists in the root directory of the boot drive, ELF-DOS runs it automatically, before showing you the first prompt. See "Writing Batch Files" below for what a batch file is and how to write one.

Drives

The four drives set up at start-up are not fixed. The **MOUNT** command attaches any partition to any letter, and **UMOUNT** detaches one. Typing **MOUNT** on its own lists what is currently in use:

```
C: /> MOUNT
Drive  Unit  Partition start LBA
C:      0      2,048
```

```
D:      0      1,955,840
E:      0      4,052,992
F:      0      6,150,144
4 of 6 drives mounted.
```

Six drives can be mounted at once, and a letter is just a name: nothing stops you from calling a partition **W:** because it holds your work, or **M:** for music. To attach the second partition as **W:**:

```
C: /> MOUNT 2 W:
Mounted partition 2 as W:
```

Mounting a letter that is already in use simply re-points it. To free a slot, unmount it:

```
C: /> UMount W:
Unmounted W:
```

If you unmount the drive you are currently on, you are moved back to the drive ELF-DOS booted from.

Other devices. Some machines have more than one storage device. **MOUNT** takes an optional unit number in front of the partition to say which one: **MOUNT 1 2 W:** means "unit 1, partition 2, as **W:**". Left out, the unit is 0 - the device ELF-DOS booted from, and the only one on most machines.

Units run from 0 to 7, so up to eight devices can be attached to the machine. That is a separate limit from the six drive letters: the six is how many partitions may be *in use at once*, drawn from any of the eight devices in any combination. Which units exist depends on the machine's ROM - a ROM that supports only one device ignores the number, so on those machines only unit 0 is meaningful.

Whole devices. A floppy disk has no partition table; the file system starts at the very beginning. Use partition number 0 for that: **MOUNT 1 0 A:** mounts all of unit 1 as **A:**. (**A:** and **B:** are free to use for this, exactly as you would expect, though nothing forces it.)

One drive cannot be unmounted: the one ELF-DOS booted from, because the commands themselves live there. **MOUNT** and **UMOUNT** both refuse.

ELF-DOS reads FAT16 volumes only. **MOUNT** checks before attaching anything and will tell you if a disk is FAT12 - common on floppies - rather than attaching it and misreading it.

Typing Commands

A command line is the command's name, followed by any arguments it needs, separated by spaces. For example:

```
TYPE readme.txt
```

File names. A file or directory name can be typed on its own (meaning "in the current directory"), or with a path in front of it:

- `/cfg/env.dat` - an absolute path, starting from the root of the current drive.
- `D:/cfg/env.dat` - an absolute path on a specific drive.
- `../backup` or `cfg/env.dat` - a path relative to the current directory.

Quoting and spaces. If a file name contains a space, put the whole path in quotes so it is treated as one argument instead of two:

```
TYPE "my notes.txt"
```

Arguments with special characters may be delimited by double quotes ("`...`") or single quotes ('`...`'). The difference is what happens to a backslash inside them. Inside double quotes, and outside of quotes entirely, a backslash followed by a character is taken literally - `\` becomes a plain `"`, `\\` becomes a single `\`, and becomes a plain space that will not split the argument. Inside single quotes, nothing is special; every character between the quotes, including a backslash or a double quote, is used exactly as typed.

Wildcards. The characters `*` and `?` can be used in a file name to match more than one file, the same as in MS-DOS. `*` matches any run of characters; `?` matches any one character. Wildcards are understood by `DIR`, `LS`, `COPY`, `MOVE`, `DEL`, `ATTRIB`, `XCOPY`, and `YS`. If a wildcard does not match anything, the command is given the text exactly as typed instead.

How ELF-DOS finds a command. When you type a name on its own, ELF-DOS looks for it in `/bin` on the drive it booted from. Those are the system commands, and nothing can displace them - a `/bin` on some other disk you have mounted cannot quietly replace `COPY` or `DEL` with its own version.

If the name is not a system command, ELF-DOS then looks in each directory listed in the `PATH` environment variable, in order, and runs the first match:

```
C:/> EXPORT PATH=D:/tools;E:/games
```

Separate directories with a semicolon. `PATH` starts out unset, which simply means only the system commands are found. Because the system commands are searched first, a mistake in `PATH` can never leave you unable to type `EXPORT` to fix it.

To run a program in the current directory, put `./` in front of its name:

```
C:/> ./mygame
```

That skips the search entirely and runs exactly the file you named, which is also how you run something whose name happens to match a system command. Any name containing a `/` works this way - `../tools/build` runs that file and nothing else.

If a command name is not found anywhere, ELF-DOS says **Bad command or file name**. That message is about the command itself; a message like **File not found** comes from a command that ran and could not find one of *its* arguments.

Substitutions. A few special sequences are replaced with a value before a command line runs:

Type this	You get
%ERRORLEVEL%	The exit code of the command that just finished.
%0 through %9	Inside a batch file, the file's own command-line arguments (%0 is the batch file's own name).
\$NAME or \${NAME}	The value of environment variable NAME (see "Environment Variables" below). An unset variable is replaced with nothing.

These are only replaced outside of quotes, or inside double quotes. Put text in single quotes if you want a **%** or **\$** left alone.

Editing the Command Line

While typing a command, you can move around and fix mistakes without retyping the whole line:

Key	Does this
Left / Right arrow, or Ctrl-B / Ctrl-F	Move the cursor one character.
Home / End, or Ctrl-A / Ctrl-E	Jump to the start / end of the line.
Backspace	Delete the character before the cursor.
Delete, or Ctrl-D	Delete the character under the cursor.
Up / Down arrow	Recall an earlier command, or step back to a later one.

These same keys also work inside the **EDLIN** text editor.

Every command you type is saved to a history file (**history.dat**, on the boot drive), and this history is what the Up and Down arrows recall. The history file trims itself automatically once it grows large, so it never needs to be cleaned out by hand.

Redirecting Input and Output

A command's output can be sent to a file instead of the screen, and a command that reads input can be given a file to read from instead of the keyboard:

DIR > listing.txt	Send DIR's output to listing.txt (replacing it)
DIR >> listing.txt	Same, but add to the end of the existing file
SOMECOMMAND < input.txt	Read input from input.txt instead of the keyboard

The special name **NUL** can be used on either side. **> NUL** throws output away; **< NUL** acts like an empty file that ends immediately. This works the same way it does in MS-DOS and Unix.

The **>**, **>>**, and **<** symbols can appear anywhere on the command line, and a space before the file name is optional (**>file** and **> file** both work). Redirection also works inside batch files.

The Command Reference

In the tables below, an argument in **<angle brackets>** is required; one in **[square brackets]** is optional. **...** means "one or more, repeated."

Looking at files and directories

Command	Usage	What it does
DIR	DIR [path...]	Lists a directory. With no arguments, lists the current directory. Given one or more files or wildcard matches, shows information about each one instead.
LS	LS [-laF] [path...]	Lists a directory, Unix-style. Plain columns by default. -l gives a detailed, one-line-per-entry listing. -a includes hidden files. -F marks directories with a trailing / . Options can be combined (-lF) or separate, and go anywhere on the line.
CD	CD <path>	Changes the current directory. CD .. goes up one level; CD / goes to the root.
PWD	PWD	Prints the full path of the current directory.
STAT	STAT <path>	Shows a file or directory's type, size, first cluster, and the date and time it was last written.
ATTRIB	ATTRIB [+H -H] <path...>	Shows or changes a file's hidden attribute. With no +H/-H , shows whether each file is hidden. +H hides it; -H unhides it.
LABEL	LABEL [drive:] [text -d]	Shows, sets, or removes a drive's volume label. -d deletes the label.
MOUNT	MOUNT [[unit] partition] [letter:]	With no arguments, lists the drives in use. Otherwise attaches a partition to a drive letter. Partition 0 means "the whole device, no partition table," for floppies. See "Drives" above.
UMOUNT	UMOUNT <letter:>	Detaches a drive letter. The boot drive cannot be detached.
CHKDSK	CHKDSK [X:]	Checks a drive for file system problems - lost clusters, files whose size does not match their data, and damaged directory entries - and prints a summary. This is a check only; it does not repair anything.

Working with files

Command	Usage	What it does
TYPE	TYPE <filename>	Displays a text file on the screen.
LESS	LESS <filename>	Pages through a file, forward AND backward, by screen or by line. SPACE/F/PgDn = next page, B/PgUp = previous page, g = go to the top, G = go to the end; typing a number first jumps there -- 50g (or 50G; either letter works) goes to line 50, and a number past the end of the file shows the last page, the same as a bare G; Down-arrow/J/Ctrl-N/Ctrl-E = down one line, Up-arrow/K/Ctrl-P/Ctrl-Y = up one line (moving up or back past the point where LESS has any recorded history -- e.g. right after G, g, or a search match -- scans backward through the file for the true previous line/page instead of just stopping); / = search forward (case-sensitive), N = repeat the last search, Q = quit. A search pattern may contain escape sequences -- \\n, \\r, \\t, \\0, \\\\ for a literal backslash, and \\xHH for any byte by hex value (exactly two digits) -- so you can search for tabs, line endings, or arbitrary binary bytes, and a pattern may span a line boundary. A backslash followed by anything else is reported as an error rather than searched for literally.
HEXDUMP	HEXDUMP <filename>	Shows a file's raw bytes, in hexadecimal and as text, side by side.
COPY	COPY [-y] <source> <destination>	Copies one file to another name, or one or more files into a directory. -y skips the "overwrite?" prompt.
MOVE	MOVE <source> <destination>	Moves or renames one or more files, the same way COPY takes its arguments.
REN	REN <path> <newname>	Renames a file or directory. It must stay in the same directory — use MOVE to move it elsewhere.
DEL	DEL <filename...>	Deletes one or more files. It will not delete a directory — use RD for that.
TOUCH	TOUCH <filename...>	Updates a file's last-modified time to right now, without changing its contents. It will not create a new file.
XCOPY	XCOPY [-s] [-e] [-y] [-d] [-c] <source> <destination>	Copies files, and optionally whole directory trees. -s includes subdirectories. -e keeps empty subdirectories that get copied along the way. -d only copies files that are newer than what's already at the destination. -c keeps going if one file fails, instead of stopping. -y skips overwrite prompts.

Directories

Command	Usage	What it does
MD	MD <path>	Makes a new, empty directory. The directory above it must already exist.

Command	Usage	What it does
<code>RD</code>	<code>RD <path></code>	Removes an empty directory. This will fail if the directory has files in it.

Environment Variables

Environment variables are simple `NAME=VALUE` pairs that programs and batch files can read. They are persistent across reboots.

Command	Usage	What it does
<code>EXPORT</code>	<code>EXPORT [name[=value]...]</code>	With no arguments, lists every variable. <code>EXPORT NAME=VALUE</code> sets one; <code>EXPORT NAME</code> sets the variable to an empty string. Use <code>UNSET</code> to delete a variable.
<code>PRINTENV</code>	<code>PRINTENV [name...]</code>	With no arguments, lists every variable and its value. Given one or more names, prints just those values.
<code>UNSET</code>	<code>UNSET name...</code>	Removes one or more variables.

Date, time, and the system

Command	Usage	What it does
<code>DATE</code>	<code>DATE [MM/DD/YYYY]</code>	Shows or sets the date.
<code>TIME</code>	<code>TIME [HH:MM[:SS]]</code>	Shows or sets the time. Seconds are optional.
<code>BAUD</code>	<code>BAUD <rate></code>	Sets the console's baud rate (300 through 57600).
<code>VER</code>	<code>VER</code>	Prints the ELF-DOS version.
<code>REBOOT</code>	<code>REBOOT</code>	Restarts the computer without turning it off.
<code>MON</code>	<code>MON</code>	Drops into the built-in ROM monitor.
<code>SYS</code>	<code>SYS <kernel-full.bin></code>	Installs a new copy of ELF-DOS onto the boot drive.
<code>CLS</code>	<code>CLS</code>	Clears the screen.

Sending and receiving files

These commands transfer files over the serial port, to and from another computer running matching transfer software.

Command	Usage	What it does
<code>MR</code>	<code>MR [-u -b] [-v] [<destination>]</code>	Receives one or more files (the host decides how many). No destination given, or an existing directory, receives everything into that directory (or the current one) under each file's own name; any other name saves only the first file received, under that exact name. <code>-v</code> prints per-file progress messages; without it, <code>MR</code> stays silent until its final summary.

Command	Usage	What it does
MS	MS [-u -b] [-v] <filename> [filename...]	Sends one or more files, using the MAX protocol. Filenames may contain */? wildcards. -v prints per-file progress messages; without it, MS stays silent until its final summary.
YR	YR [-u -b] [-y] [-v]	Receives one or more files in a batch, using the YMODEM protocol. -y overwrites an existing file without asking -- strongly recommended whenever the console and transfer port might share a wire (see below), since the interactive overwrite prompt otherwise has nowhere else to go. -v prints per-file progress messages; without it, YR stays silent until its final summary.
YS	YS [-u -b] [-k] [-y] [-v] <filename...>	Sends one or more files in a batch, using the YMODEM protocol. Filenames may contain */? wildcards. -k sends 1024-byte data blocks instead of the default 128-byte ones. -y is accepted for symmetry with YR but does nothing -- YS never writes a local file, so there's nothing for it to protect. -v prints per-file progress messages; without it, YS stays silent until its final summary.

All four use the board's normal console I/O by default, so a transfer follows whatever serial port (or other device) the console is already using, with no port selection needed. If the board has more than one serial port, -u (the disk-board UART) or -b (the onboard, bit-banged port) sends the WHOLE transfer over that specific port instead, regardless of what the console currently is. This is useful for running a transfer on a different port than the one you're typing commands on -- for example, watching debug output on the console while a transfer runs on the other port, or driving a transfer from a separate program instead of a terminal session.

All four default to printing nothing until they're done, since their console and the transfer's own wire are frequently the same physical connection -- progress text interleaved with protocol bytes on a shared line can be misread as part of the transfer itself (a real failure mode: MS's own "Sent <name>." message, if it leaked onto a shared wire, could be misread by the far end as the start of the next chunk's length field). -v opts back into per-file progress messages when the console and transfer port are genuinely separate (or you're willing to accept the risk on a shared one). YR's own overwrite prompt is the one thing -v/quiet-by-default can't cover -- it needs a real answer from a real person, so -y (skip the prompt entirely) is the actual fix for that specific case on a shared wire, not -v.

MR/MS's companion on the host side is max-xfr (max-xfr -s <files...> to send, max-xfr -r [<destination>] to receive) -- matching syntax to MR/MS in each direction. max-xfr also takes a -v (progress) and -d <microseconds> (a small delay before each byte it writes, including its own acks) -- needed on some bit-banged links, since the ELF-DOS side's serial receive has no buffering of its own and can miss a byte that arrives before it's actually polling for one.

Editing text

EDLIN is a line editor, similar to the classic MS-DOS EDLIN:


```
EDLIN [filename]
```

Type **EDLIN** alone to start with an empty file, or **EDLIN filename** to open an existing one. **EDLIN** shows a ***** prompt and works one line at a time. The commands it understands are:

Command	Does this
L or P	Lists lines. L n or L n1,n2 lists a specific line or range.
A plain number	Shows that line, then lets you type a replacement (press Enter to leave it unchanged).
I	Inserts new lines before the current line. Type . alone on a line to stop inserting.
A	Appends new lines at the end of the file, the same way I works.
D n or D n1,n2	Deletes a line or range of lines.
S text	Searches forward for text .
R, C, M	Replace, copy, or move lines.
T filename	Reads another file in at the current line.
W [filename]	Saves the file without exiting.
E [filename]	Saves the file and exits.
Q	Quits without saving.

Wherever a line number is expected, you can also type **.** for the current line, **\$** for the last line, or **#** for one past the last line.

Other small utilities

Command	Usage	What it does
ARGS	ARGS [anything...]	Prints back each argument on its own line. Useful for checking exactly how the shell split up a command line.
ECHO	ECHO [-n] [text...]	Prints the given text. -n leaves off the final newline. ECHO ON and ECHO OFF turn batch-file line echoing on or off (see below); ECHO alone reports which one is currently in effect.

Writing Batch Files

A batch file is a text file of commands, run one after another, the same way you would type them yourself. Its name must end in **.bat**. You can create one with **EDLIN** or by transferring a text file from another computer. To run it, type its name, exactly like any other command:

```
C:/> myscript.bat
```

A first batch file

Here is a small batch file that lists a directory and shows the date:

```
DIR /cfg  
DATE
```

Save this as `show.bat` and run it by typing `show.bat`. ELF-DOS shows each line as it runs, the same as if you had typed it yourself, so the screen looks like this:

```
C:/> show.bat  
DIR /cfg  
... the directory listing ...  
DATE  
... the current date ...
```

Controlling what gets shown

Seeing every line echoed is often more than you want. `ECHO OFF`, on a line by itself, turns this off for the rest of the file:

```
ECHO OFF  
DIR /cfg  
DATE
```

Now only the commands' own output appears, not the commands themselves. You can also silence just one line, even with echo on, by putting `@` in front of it:

```
@DIR /cfg
```

A blank line, or a line starting with `REM`, does nothing - use `REM` to leave yourself notes:

```
REM This script backs up the configuration directory.
```

Using arguments

A batch file can take its own arguments, the same way any command does. Inside the file, `%1` is the first argument, `%2` is the second, and so on; `%0` is the batch file's own name. For example, `backup.bat`:

```
ECHO OFF
COPY %1 /backup
ECHO Done.
```

Running `backup.bat readme.txt` copies `readme.txt` into `/backup`, then prints `Done..`

`%ERRORLEVEL%` works the same way, standing for the exit code of the command that just ran (0 usually means success). You can check it right after a command to see whether it worked:

```
COPY %1 /backup
IF %ERRORLEVEL%==0 ECHO Copy succeeded.
```

Testing conditions with IF

`IF` runs a command only when something is true:

```
IF EXIST %1 ECHO The file exists.
IF NOT EXIST %1 ECHO The file is missing.
```

`IF string1==string2 command` compares two pieces of text instead, and runs `command` only if they match exactly:

```
IF %1==readme.txt ECHO That is the readme.
```

There is no `ELSE`. Write the opposite condition on its own line instead, as shown above with `EXIST/NOT EXIST`.

Jumping around with GOTO

`GOTO label` skips ahead or back to a line starting with `:label`. This is how a batch file can skip a step, or loop. Putting it together with `IF`, here is a batch file that copies a file only if it exists, and explains itself either way:

```
ECHO OFF
IF NOT EXIST %1 GOTO missing

COPY %1 /backup
ECHO Copied %1 to /backup.
GOTO end

:missing
ECHO %1 does not exist.
```

```
:missing  
ECHO %1 does not exist -- nothing copied.  
  
:end
```

If the label named in a **GOTO** doesn't exist anywhere in the file, ELF-DOS prints **Label not found.** and stops the batch file right there.

A few things to keep in mind

- If a command inside the batch file fails, or isn't found, ELF-DOS shows the usual error message and simply moves on to the next line - it does not stop the whole batch file.
- A batch file cannot run another batch file from inside itself.
- If a file named **autoexec.bat** exists in the root directory of the boot drive, it runs automatically every time the computer starts, before the first prompt is shown - a good place to put anything you want set up every time (setting the date, changing to a working directory, and so on).

A Few Notes About the File System

- ELF-DOS uses FAT16, the same file system used by early MS-DOS. Long, lowercase file names are supported normally, alongside the traditional eight-character-name-plus-three-character-extension form.
- The root directory of a drive has no **.** or **..** entries. A path starting with **.** cannot be used to refer to the root - use a plain name, or a path starting with **/**, instead.